

Software Dependability Report

Carmine Iemmino

ACM Reference Format:

Carmine Iemmino. 2025. Software Dependability Report. In . ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 INTRODUCTION

1.1 Structure of the report

This report contains an analysis of the Apache Commons Configuration Java library in regards to its dependability attributes. It is divided into the following sections:

- **Section 1 - Introduction:** Describes what the report contains, what type of library is analyzed and the evaluation criteria.
- **Section 2 - Quality Analysis:** Examines the quality attributes of the library, leveraging Sonar Cloud to identify code smells, bugs, and other maintainability issues. It also discusses refactoring strategies to improve code quality.
- **Section 3 - Containerization:** Explores the process of packaging the library into a containerized environment.
- **Section 4 - Testing:** Analyzes the testing strategies applied to the library. It includes discussions on test coverage, mutation testing, automatic test case generation using Randoop and GitHub Copilot, and performance testing with JMH.
- **Section 5 - Vulnerabilities:** Investigates the library's vulnerabilities using Sonar Cloud, FindSecBugs, and Dependency Check. Discusses identified issues and the steps taken to mitigate or fix them.
- **Section 6 - Conclusions:** Summarizes the findings of the analysis, highlighting key insights into the library's dependability and suggesting areas for future improvement.

1.2 What is commons-configuration?

Apache Commons Configuration is a robust library within the Apache Commons project that provides a unified interface for managing configuration settings in Java applications. It supports various configuration formats, including properties files, XML, INI files, and more, enabling developers to handle application configurations seamlessly. As of now, the Apache Commons Configuration project has garnered contributions from 51 individuals. The project's repository on GitHub¹ has accumulated over 4,376 commits, reflecting ongoing development and maintenance efforts. The project has also seen a number of pull requests, with 5 currently open and 510

¹<https://github.com/apache/commons-configuration>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, July 2017, Washington, DC, USA

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

closed, indicating active participation from the developer community. The project team comprises members with direct access to the source code, actively evolving the codebase, and contributors who enhance the project through patches and suggestions. This collaborative effort ensures that Commons Configuration remains a reliable and up-to-date tool for managing configurations in Java applications.

1.3 Project evaluation criteria

All evaluation criteria for the project have been satisfied:

- **Buildability:** The project is buildable locally and CI/CD through GitHub Actions.
- **Quality Analysis:** The project has been analyzed through Sonar Cloud and most of high priority issues refactored. A rationale for the skipped refactorings is provided.
- **Containerization:** An image for the project has been created, and its container can be run as a Java application.
- **Code coverage:** Code coverage is analyzed through JaCoCo.
- **Mutation testing:** A mutation testing campaign has been conducted to assess the test cases of the project using PiTest.
- **Automatic test generation:** Code coverage has been improved through automatic test generation with GitHub CoPilot.
- **Performance testing:** Performance tests have been implemented using JMH.
- **Vulnerabilities:** Vulnerabilities are discovered, analyzed and some fixed with FindSecBugs and Dependency Check.

2 QUALITY ANALYSIS

2.1 Introduction

To begin the analysis, the project's original repository was forked². The forked repository was then cloned locally and opened in IntelliJ IDEA. Using Apache Maven, the project was successfully built, tested, and packaged. The CI/CD pipeline has been configured to target the latest LTS version of Ubuntu, with JDK 23 as the reference platform.

2.2 Sonar Cloud

The project has been analyzed using the Sonar Cloud service. The service found over 3000 issues. The high priority issues are 108, of which 55 related to intentionality, 38 adaptability and 15 to consistency.

- **Remove usage of generic wildcard type (36 occurrences):** Generic wildcard types reduce type safety. Replacing them with specific types improves code readability and prevents potential runtime errors.
- **Cognitive complexity (14 occurrences):** High cognitive complexity indicates that certain portions of code are difficult to understand and maintain. Refactoring is required to simplify the logic and enhance readability.

²<https://github.com/SugarStoneMaster/commons-configuration>

- **Missing assertions in test cases (13 occurrences):** Some test cases lack assertions, making them ineffective in verifying expected outcomes. Adding assertions enhances test reliability.
- **Remove clone implementation (13 occurrences):** Cloning is often unnecessary and can lead to issues with object states. Removing the clone implementation reduces maintenance complexity.
- **Define constant instead of using literal (8 occurrences):** Repeated literals should be replaced with constants to improve maintainability and reduce the risk of errors.
- **Empty methods (8 occurrences):** Methods without implementations serve no purpose and should either be implemented or removed to clean up the codebase.
- **Transient or serializable (7 occurrences):** Fields in serializable classes should be explicitly marked as transient or properly handled to ensure correct serialization behavior.
- **Use already defined constant (2 occurrences):** Duplicate literals should be replaced with already defined constants to improve consistency and reduce redundancy.
- **Rename field (1 occurrence):** Field names may be unclear or misleading. Renaming them improves code clarity and self-documentation.
- **Rename method (1 occurrence):** Method names may not accurately reflect their functionality. Renaming methods enhances understandability.
- **Add some tests to this class (1 occurrence):** A test class lacked proper test methods, which should be added to ensure functionality is verified and defects are minimized.
- **Other (4 occurrences):** Miscellaneous minor issues that do not fall into the categories above but still require attention.

2.3 Refactoring

The goal was to refactor all high priority issues or to provide a rationale for the skipped refactorings.

- **Remove Usage of Generic Wildcard Type:** All 36 occurrences of this issue were reviewed and ultimately skipped for refactoring. Resolving these would have required a deep understanding of the complex codebase and its dependencies, which was outside the project's scope. Additionally, such changes posed a high risk of introducing regressions or breaking functionality, particularly in areas with limited test coverage. Given the time constraints and the relatively low impact of these issues on the overall functionality and reliability of the project, efforts were focused on addressing more critical areas.
- **Cognitive Complexity:** Of the 14 identified cognitive complexity issues, 13 have been successfully refactored to improve code readability and maintainability. However, one issue with a particularly high complexity score of 39 was left unchanged. Refactoring it has been tried but with no results.
- **Missing assertions in test cases:** All 13 occurrences have been refactored, adding assertions where needed.
- **Remove Clone Implementation:** All 13 occurrences of this issue were not refactored due to the extensive changes required to replace the clone() method with a copy constructor.

Such a change would necessitate significant modifications to the usage patterns throughout the codebase, potentially introducing instability and requiring substantial retesting. Additionally, the existing clone() implementations are functional and well-integrated into the project, minimizing the immediate need for this refactoring.

- **Define constant instead of using literal:** All 8 occurrences have been refactored, defining constants where needed.
- **Empty methods:** All 8 occurrences have been refactored, putting comments in the empty methods.
- **Transient or serializable:** All 7 occurrences have been refactored, making attributes transient or defining custom readObject and writeObject.
- **Define constant instead of using literal:** All 8 occurrences have been refactored, defining constants where needed.
- **Use already defined constant:** All 2 occurrences have been refactored, using already defined constants where needed.
- **Rename field:** The single occurrence of this issue was not refactored due to compatibility constraints with JApiCmp. Renaming the field would break backward compatibility and impact API stability, which is critical for maintaining seamless integration with dependent projects.
- **Rename method:** The single occurrence of this issue was not refactored due to compatibility constraints with JApiCmp. Renaming the method would break backward compatibility and impact API stability, which is critical for maintaining seamless integration with dependent projects.
- **Add Some Tests to This Class:** The single occurrence of this issue was not addressed as the class in question is a utility class specifically designed to support other test classes. Its functionality is indirectly verified through the tests that rely on it, making additional standalone tests redundant.

3 CONTAINERIZATION

In order to simplify the deployment process and ensure the application runs in a consistent environment, a Docker image has been created and pushed on DockerHub³. The Dockerfile has been configured to use an official Java runtime (openjdk:23-jdk-slim) as a lightweight parent image. The application JAR file, produced by the Maven build, has been copied into the container along with a default config.properties file. This setup ensures the application has a built-in configuration by default.

Inside the Docker container, the Java application is launched by invoking:

```
CMD ["java", "-jar", "app.jar"]
```

This command instructs the container to run the main class that is packaged in app.jar. The main class (MainApp) loads properties from /app/config.properties (or another path if specified by the CONFIG_FILE_PATH environment variable) and demonstrates various features of the Apache Commons Configuration library (e.g., loading properties, interpolation, list properties, and combined configurations).

To run the container, the image can be built via:

```
docker build -t myapp:latest .
```

³<https://hub.docker.com/repository/docker/sugarstonemaster/commons-configuration/general>

and then started with:

```
docker run -rm myapp:latest
```

If no extra configuration is mounted or provided, the application automatically uses the default file shipped within the image. Should an alternate configuration be necessary, the environment variable `CONFIG_FILE_PATH` can be overridden, and a volume can be mounted accordingly, enabling a customized configuration at run-time.

4 TESTING

4.1 Introduction

Commons Configuration includes an extensive test suite implemented using JUnit. After executing the test suite via Maven, the results indicate that all 2953 tests passed successfully, with no failures. However, 2 tests were skipped during the execution.

4.2 Coverage

The project supports coverage analysis via the JaCoCo Maven plugin, which is configured to generate coverage reports during the build process. These reports provide insights into the extent of code exercised by the test suite, helping identify areas that may need additional testing to ensure comprehensive validation.

- The overall instruction coverage is **86%**, with 6,391 instructions missed out of 48,153 and the overall branch coverage is **82%**, with 756 branches missed out of 4,367. This indicates a solid testing effort but leaves some room for improvement in specific areas.
- The `org.apache.commons.configuration2.plist` package, which appears to be one of the more complex components, has relatively low instruction coverage at **64%** and branch coverage at **54%**. This suggests that additional tests should be added to ensure robustness in this area.
- Packages such as `org.apache.commons.configuration2.tree` and `org.apache.commons.configuration2.builder.combined` stand out with excellent instruction and branch coverage at **97%**. This highlights their thorough testing and reliability.
- Smaller packages such as `org.apache.commons.configuration2.builder.fluent` and `org.apache.commons.configuration2.sync` achieve **100%** coverage, showcasing their simplicity and comprehensive testing.
- Out of 4,367 methods in the codebase, **3,280** are tested, leaving 1,087 methods untested. A focus on these untested methods could further increase coverage and reliability.

The coverage report demonstrates a robust test suite, particularly for critical and high-complexity packages. However, areas with lower coverage, such as `plist` and `resolver`, require additional tests to improve the overall quality and reliability of the project.

4.3 Mutation Testing

A mutation testing campaign was conducted using PiTest with default configurations. The campaign aimed to evaluate the robustness and quality of the test suite by introducing controlled mutations into the codebase and measuring the ability of the test

suite to detect these mutations. Below are the key observations from the mutation report:

- The overall **line coverage** reached **88%**
- The **mutation coverage** was reported at **79%**
- The **test strength** was high, reaching **91%**
- The core package `org.apache.commons.configuration2` achieved **92%** test strength and **84%** mutation coverage. Although well-tested, some mutations escaped detection, suggesting room for improvement in edge cases.
- The `org.apache.commons.configuration2.builder` and `org.apache.commons.configuration2.builder.combined` packages stood out with impressive **97%** test strength and mutation coverage above **92%**, demonstrating strong test coverage in these areas.
- The `org.apache.commons.configuration2.plist` package showed relatively weak performance, with **44%** mutation coverage and only **70%** test strength. This package would benefit from additional tests to improve reliability.
- The `org.apache.commons.configuration2.resolver` package showed relatively weak performance, with **43%** mutation coverage and only **51%** test strength. This package would benefit from additional tests to improve reliability.

Overall, the mutation testing campaign demonstrated a robust and effective test suite for the majority of the codebase. However, some areas, particularly the `plist` and `resolver` packages, require additional attention to enhance their mutation detection capabilities and ensure higher test reliability.

4.4 Automatic generation of test cases

The generation of test cases focused on the weak packages that have been found through JaCoCo and PiTest: `plist` and `resolver`

4.4.1 Randoop. While Randoop is a powerful framework for automated test generation, it proves unsuitable for both the `plist` and `resolver` packages for several key reasons:

- **Dependence on External or Structured Inputs:** Both packages require well-defined configuration files (e.g., `.plist`) or XML catalogs/URLs. Randoop's random input generation cannot reliably produce valid external resources or structured content, leading to frequent invalid or meaningless tests.
- **Complex State and Interactions:** Many operations (loading configurations, resolving entities, modifying settings) involve multi-step sequences with interdependent internal states. Random calls from Randoop are unlikely to mirror realistic usage or maintain consistent states across method calls.
- **Manual Validation Overhead:** Even if Randoop generated tests, considerable manual effort would be required to validate, refine, or interpret them (especially given custom exceptions, logging, or file-based I/O), undermining the automation benefits.
- **Domain-Specific Structure:** Methods in both packages expect highly structured data (like hierarchical configuration trees or public/system IDs), making it difficult for random

test generation to produce inputs that conform to the domain's requirements.

Possible Future Use of Randoop: Although Randoop is not beneficial for classes that depend on file-based configurations or complex state, it might be applied in the future to more self-contained parts of these packages. In particular, individual classes with straightforward logic and no reliance on external resources or multi-step interactions could be tested with random input generation to increase coverage for their simpler methods.

4.4.2 GitHub Copilot. GitHub Copilot was effectively utilized to enhance the coverage of the two packages with the lowest test coverage: `resolver` and `plist`. In the `resolver` package, line coverage increased significantly from 75% to 88%, while branch coverage improved from 64% to 83%. Similarly, in the `plist` package, line coverage rose from 64% to 67%, and branch coverage slightly improved from 54% to 55%. These enhancements were achieved through the addition of **15 new test cases**, bringing the total number of tests in the project to **2,968**. A subsequent run of PiTest has shown a slight increase in both line coverage and mutation coverage: line coverage rose from 88% to 89%, and mutation coverage improved from 79% to 80%. By contrast, test strength declined marginally from 91% to 90%. These changes indicate that new or refined tests are effectively exercising more code segments and neutralizing more mutants overall, thereby boosting line coverage and mutation coverage. However, the small drop in test strength suggests that, relative to newly tested or modified lines of code, a proportion of newly introduced mutants remained undetected. As a result, the ratio of killed mutants to total mutants decreased slightly, causing the observed dip in test strength.

4.5 Performance Testing

Performance testing was conducted locally on a macOS MacBook M2 Pro using the **Java Microbenchmark Harness (JMH)** to measure the efficiency and throughput of specific methods in the Apache Commons Configuration library. JMH is well-suited for microbenchmarking as it provides precise, reproducible results while accounting for JVM optimizations such as Just-In-Time (JIT) compilation.

The following methods were selected for benchmarking, focusing on their execution time and throughput under various workloads:

- **getString(String key)** (class `PropertyListConfiguration`): This method retrieves the value associated with a given key. It is frequently used in real-world configurations and its efficiency directly impacts application performance. Uses three different sized sample configuration files in order to evaluate performance on different workloads.
- **write(Writer out)** (class `PropertyListConfiguration`): This method serializes the configuration data to an output stream. The performance of this method is crucial when persisting large or complex configurations. Uses three different sized sample configuration files in order to evaluate performance on different workloads.
- **to(Class<?> cls, Object value, DefaultConversionHandler convHandler)** (class `PropertyConverter`): Benchmarked in two versions:
 - **Original version** with a cognitive complexity of 44.

- **Refactored version** with reduced cognitive complexity, now under 15, aimed at improving maintainability.

The benchmarks use parameters to represent both the target class for conversion and the input value being converted. The `targetClassName` parameter specifies common data types such as `Integer`, `Long`, `BigInteger`, and `BigDecimal`. The `inputValue` parameter provides representative values like 42, 2147483647, and other numeric values to simulate real-world conversion tasks.

- **unescapeJava(String str, boolean jupCompatible)** (class `PropertiesConfiguration`): Benchmarked in two versions:

- **Original version** with a cognitive complexity of 20.
- **Refactored version** with reduced cognitive complexity, now under 15, aimed at improving maintainability.

The benchmarks use input strings that simulate various levels of complexity to evaluate the method's performance under diverse conditions. These include strings with escape sequences such as `\n` for newlines, `\t` for tabs, and Unicode characters like `\u0041` (representing 'A'). Inputs range from simple strings with a few escape sequences to larger strings with repeated escape patterns and Unicode sequences to test the method's handling of complex cases.

4.5.1 Results.

1. Performance of getString(String key). This method retrieves a property value based on a specified key. Benchmarks were performed using three file sizes (`sampleSmall.plist`, `sampleMedium.plist`, `sampleLarge.plist`), measuring throughput in operations per millisecond (ops/ms):

- **sampleSmall.plist:** Throughput was around **4800 ops/ms**, reflecting highly efficient lookups in smaller configurations.
- **sampleMedium.plist:** The throughput dropped to about **3883 ops/ms**, indicating that additional data in the medium file does slow retrieval somewhat.
- **sampleLarge.plist:** Interestingly, throughput remained roughly **3953 ops/ms**, showing that adding more data did not further degrade performance in this case.

2. Performance of write(Writer out). Here, the configuration data are serialized to an output stream:

- **sampleSmall.plist:** The throughput reached **633 ops/ms**, confirming quick serialization for minimal data.
- **sampleMedium.plist:** Performance decreased to **201 ops/ms**, highlighting the increased overhead for more properties.
- **sampleLarge.plist:** The throughput further dropped to about **60 ops/ms**, underscoring the expensive nature of formatting and writing large configurations.

Overall, serialization scales poorly with file size, as the process involves traversing and re-formatting every property.

3. Performance of to(Class<?> cls, Object value, DefaultConversionHandler convHandler). This method was benchmarked in two variants:

- **Original version**, with a cognitive complexity of 44.
- **Refactored version**, reduced below 15 to improve maintainability.

Both variants parse numeric values (e.g., 42, 2147483647, 314159265, 0) into Integer, Long, BigInteger, or BigDecimal. Key observations include:

- **Integer & Long:** Parsing consistently exceeds **35–40k ops/ms**, even for large integers like 2147483647, indicating low overhead for integral conversions.
- **BigInteger:** Yields noticeably lower throughput (**6–12k ops/ms**), reflecting higher costs of arbitrary-precision arithmetic.
- **BigDecimal:** Ranges around **20–29k ops/ms**, slower than Integer/Long but typically outperforming BigInteger.
- **Original vs. Refactored:** Performance remains almost identical across both versions. The refactoring significantly eases code complexity (from 44 down to < 15) without sacrificing speed.

4. *Performance of unescapeJava.* This method unescapes Java-specific escape sequences such as \n, \t, and \uXXXX. Its original implementation had a cognitive complexity of 20, which was reduced below 15 in a refactored version. The benchmark tested different string lengths and a boolean parameter `jupCompatible`, measuring throughput in ops/ms:

- **Short Strings:** Both refactored and original methods exceed **2700 ops/ms**, with negligible impact from `jupCompatible`.
- **Moderate Strings:** Throughput hovers around **1800–2000 ops/ms**, again showing minor differences based on the boolean flag.
- **Large/Repetitive Unicode:** Performance drops significantly (e.g., **136–137 ops/ms** in the refactored vs. **65–68 ops/ms** in the original). The refactored method actually outperforms the original by about a factor of two, likely due to streamlined logic for handling successive escape sequences.

Overall, the refactored version not only improves maintainability but also retains or improves throughput, especially when decoding large strings of escaped Unicode characters.

5 VULNERABILITIES

5.1 Sonar Cloud

SonarCloud identified 5 low-priority security hotspots: 4 were related to debug features (stack trace printing) and 1 was associated with setting JavaBean properties. These vulnerabilities were not addressed because they pose minimal security risks in the current context. The debug-related issues are only relevant in development or testing environments, where security exposure is limited. The JavaBean property setting issue occurs in a controlled context where the inputs are validated and managed, making exploitation unlikely. Fixing these issues would have introduced unnecessary complexity without a significant improvement in security.

5.2 FindSecBugs

FindSecBugs found 24 vulnerabilities, divided in:

- **LDAP Injection:** 5 occurrences were flagged for potential vulnerability to LDAP injection, where user input could be improperly sanitized or validated.

- **File Location Vulnerabilities:** 10 occurrences read file locations potentially specified by user input, which may lead to directory traversal or unauthorized file access.
- **SQL Injection:** 1 occurrence flagged for vulnerability to SQL injection, where crafted inputs could manipulate database queries.
- **Web Server Request Vulnerabilities:** 4 occurrences where web server requests might expose internal services or the filesystem, posing risks of unauthorized data exposure.
- **XML External Entity Attacks:** 3 occurrences where `DocumentBuilder.parse(InputSource)` was flagged for vulnerability to XXE attacks, allowing malicious XML payloads to access sensitive resources.
- **XSLT External Entity Attacks:** 1 occurrence of `TransformerFactory.newInstance()` flagged for vulnerability to XSLT External Entity attacks, which could lead to unauthorized access or execution of malicious code.

5.3 Dependency Check

The dependency check was conducted using version 8.2.1 of the OWASP Dependency-Check tool. A total of 68 dependencies were scanned, with 41 unique ones identified. Among these, 5 dependencies were flagged as vulnerable, resulting in 13 reported vulnerabilities. Below are the details of the vulnerabilities and their respective descriptions:

- **com.squareup.okhttp:okhttp@2.7.5**
CVE-2021-0341: Improper certificate validation in the `OkHostnameVerifier` may lead to accepting a certificate for the wrong domain, causing remote information disclosure. The issue affects versions up to 2.7.5.
CVE-2023-0833: An information disclosure vulnerability via exceptions triggered by illegal header values. It could allow authenticated attackers to access sensitive information. This issue affects versions prior to 4.9.2.
- **com.squareup.okio:okio@1.6.0**
CVE-2023-3635: A denial-of-service vulnerability caused by unhandled exceptions in the `GzipSource` class while parsing malformed GZIP buffers. This issue affects versions from 0.5.0 to 1.17.6.
- **commons-jxpath:commons-jxpath@1.3**
CVE-2022-41852 (withdrawn): Originally marked for unsafe reflection due to externally controlled input to select classes or code but later withdrawn as not a valid security issue.
CVE-2022-40159 and CVE-2022-40160 (disputed): Out-of-bounds write issues reported via `'oss-fuzz'`, disputed by maintainers who stated they are invalid security issues. They still affect versions up to 1.3.
- **org.apache.hadoop:hadoop-hdfs-client@3.3.1**
CVE-2021-37404: A heap buffer overflow vulnerability in `libhdfs` that allows arbitrary code execution or denial of service when opening unvalidated file paths.
CVE-2022-25168: A command injection vulnerability in the `'FileUtil.unTar'` API due to unescaped input filenames passed to the shell. It affects versions up to 3.3.2.

CVE-2022-26612: Improper symlink resolution on Windows allows writing outside the intended directory during TAR extraction. This issue affects versions up to 3.3.1.

CVE-2021-25642: A deserialization vulnerability in the 'ZK-ConfigurationStore' allows remote attackers with ZooKeeper access to execute arbitrary commands as the YARN user.

CVE-2021-33036: A privilege escalation vulnerability that allows users with 'yarn' permissions to execute commands as the root user.

CVE-2023-26031: A relative library resolution issue in the 'container-executor' binary allows local users to gain root privileges in improperly configured YARN Secure Containers. This issue affects versions from 3.3.1 to 3.3.4.

- **original-commons-configuration2-2.11.1-SNAPSHOT.jar**

CVE-2021-37533: An FTP client vulnerability in Apache Commons Net prior to 3.9.0 trusts hosts from the PASV response by default, potentially exposing private network information. The default behavior was changed in version 3.9.0 to ignore such hosts.

5.4 Fixing the vulnerabilities

Only 4 vulnerabilities out of the 24 found by FindSecBugs were fixed. In particular it was tried to fix at least one vulnerability for each category described previously.

- **LDAP Injection:** Out of 5 occurrences, 1 was addressed by implementing input sanitization to prevent potential LDAP injection attacks. However, due to the static nature of FindSecBugs, the fix was not detected during the scan. To avoid false positives in future scans, a suppression annotation was applied for the resolved vulnerability.
- **File Location Vulnerabilities:** Among 10 detected issues related to file location vulnerabilities, 1 was fixed by sanitizing file names to prevent the risk of unauthorized access to the file system. Similar to the LDAP injection issue, FindSecBugs was unable to confirm the fix due to its static nature, necessitating the use of a suppression annotation.
- **SQL Injection:** The SQL injection vulnerability was not addressed because the issue was deeply integrated within the database configuration logic. Attempting to refactor or sanitize the code would break existing tests and potentially disrupt the functionality of the application.
- **Web Server Request Vulnerabilities:** These vulnerabilities involve web server requests potentially exposing internal services or the filesystem. While efforts were made to address them, the required changes would have caused a significant number of test failures. As the vulnerabilities are only exploitable under specific conditions, the decision was made to retain the existing implementation.
- **XML External Entity (XXE) Attacks:** Out of 3 occurrences, 1 was resolved by enabling secure processing in the XML parser. This was achieved by disabling external access to Document Type Definitions (DTDs) and external entities.
- **XSLT External Entity (XEE) Attacks:** The single occurrence of this vulnerability was successfully mitigated by enabling secure processing. External access to DTDs and stylesheets was disabled, ensuring that potentially malicious

external entities are not resolved during XSLT transformations.

Regarding the Dependency-Check analysis, 10 out of the 13 identified vulnerabilities were successfully resolved. By explicitly specifying the latest version of the hadoop-hdfs-client dependency, 9 vulnerabilities were fixed. This update also addressed vulnerabilities in the transitive dependencies, okio and okhttp. One of the remaining vulnerabilities was associated with the original jar file, which is generated before the shading process. Since the original jar is not utilized in production environments, this vulnerability was deemed low-risk and has been ignored. The vulnerabilities linked to the jxpath library have not been resolved because the library has not been maintained or updated since 2008, making it impossible to address these issues without entirely replacing the library, which would require significant rework of the codebase.

6 CONCLUSIONS

In summary, the Apache Commons Configuration library exhibits strong overall dependability, supported by a robust testing framework, ongoing development, and active community contributions. The following key insights emerged from the analysis:

- **Quality and Maintainability:** Sonar Cloud identified a substantial number of issues, many of which were addressed or given a clear rationale for skipping. Major refactoring efforts significantly improved cognitive complexity in selected methods. While certain legacy designs (e.g., generic wildcard types, clone implementations) were left untouched, these do not pose immediate functional risks.
- **Testing and Coverage:** The project's test suite demonstrates high coverage, especially in core packages, and strong mutation scores in most areas. However, packages like `plist` and `resolver` remain comparatively under-tested. Automatic test generation via GitHub Copilot yielded promising results, boosting coverage in these weaker areas by producing targeted test cases. Randoop, in contrast, proved unsuitable for classes requiring well-structured or external inputs.
- **Performance and Refactoring Gains:** Microbenchmarking revealed that refactored methods with reduced cognitive complexity maintained (or even improved) speed, underscoring that improved maintainability need not degrade performance. The tests highlighted the library's efficiency in smaller or moderate data scenarios, although certain operations (e.g., serialization of large files) scale poorly with increasing configuration size.
- **Vulnerabilities and Mitigation:** Security scanning via FindSecBugs and Dependency-Check uncovered potential injection points and outdated dependencies. Targeted fixes were applied—particularly to LDAP and file-handling vulnerabilities—while certain issues with minimal security impact remain documented but unfixed for now. Library updates addressed most transitive dependencies, mitigating a majority of reported vulnerabilities.